Tanishq Chauhan

Test ID: 450016068234845 | 7060146861 | Tanishq.120139@stu.upes.ac.in

Test Date: January 13, 2026

Computer Science	MS Excel	Software Development (Full Stack L1)	Computer Programming
85 /100	78 /100	40 /100	70 /100

Automata	Automata Fix
70 /100	100 /100

Computer Science — 85 / 100

OS and Computer Architecture	DBMS	Computer Networks
83 / 100	100 / 100	57 / 100

MS Excel — 78 / 100

MS Excel Basics	Data Analysis and Presentation
63 / 100	94 / 100

Software Development (Full Stack L1) — 40 / 100

RESTful Web Services	GIT	CSS3
100 / 100	0 / 100	0 / 100
ReactJS	**HTML5**	**Logical Ability**
33 / 100	0 / 100	60 / 100

Computer Programming — 70 / 100

Basic Programming	Data Structures	OOP and Complexity Theory
79 / 100	67 / 100	64 / 100

© 2026 SHL and/or its affiliates. All rights reserved..

Automata

70 / 100

Programming Ability	Programming Practices	Functional Correctness
60 / 100	75 / 100	65 / 100

Automata Fix

100 / 100

Logical Error	Code Reuse	Syntactical Error
100 / 100	100 / 100	100 / 100

© 2026 SHL and/or its affiliates. All rights reserved..

1 | Introduction

About the Report

This report provides a detailed analysis of the candidate's performance on different assessments. The tests for this job role were decided based on job analysis, O*Net taxonomy mapping and/or criterion validity studies. The candidate's responses to these tests help construct a profile that reflects her/his likely performance level and achievement potential in the job role

This report has the following sections:

The **Summary** section provides an overall snapshot of the candidate's performance. It includes a graphical representation of the test scores and the subsection scores.

The **Insights** section provides detailed feedback on the candidate's performance in each of the tests. The descriptive feedback includes the competency definitions, the topics covered in the test, and a note on the level of the candidate's performance.

The **Response** section captures the response provided by the candidate. This section includes only those tests that require a subjective input from the candidate and are scored based on artificial intelligence and machine learning.

The **Learning Resources** section provides online and offline resources to improve the candidate's knowledge, abilities, and skills in the different areas on which s/he was evaluated.

Score Interpretation

All the test scores are on a scale of 0-100. All the tests except personality and behavioural evaluation provide absolute scores. The personality and behavioural tests provide a norm-referenced score and hence, are percentile scores. Throughout the report, the colour codes used are as follows:

- 70 ≤ Score < 100
- 30 ≤ Score < 70
- 0 ≤ Score < 30

© 2026 SHL and/or its affiliates. All rights reserved..

2 Insights

There is no descriptive feedback for any of the tests taken by the candidate.

© 2026 SHL and/or its affiliates. All rights reserved..

3 | Response

Automata

70 / 100 Code Replay

Question 1 (Language: Java) High similarity detected : **97%**

A company called Digicomparts manufactures 52 types of unique products for laptop and desktop computers. It manufactures 10 types of laptop products and 42 types of desktop products. Each product manufactured by the company has a unique productID from a-z and A-Z. The laptop products have productIDs (a, i, e, o, u, A, I, E, O, U) while the rest of the productIDs are assigned to the desktop products. The company manager wants to find the sales data for the desktop products.

Write an algorithm to help the manager find the productIDs of the desktop products.

Scores

Programming Ability

100 / 100

Completely correct. A correct implementation of the problem using the right control-structures and data dependencies.

Programming Practices

75 / 100

Low readability, high on program structure. The source code does not follow best practices in its formatting and may contain a few redundant/improper coding constructs.

Functional Correctness

100 / 100

Functionally correct source code. Passes all the test cases in the test suite for a given problem.

Final Code Submitted — Compilation Status: Pass

```
1  import java.util.*;
2  import java.lang.*;
3  import java.io.*;
4
5  /*
6   * productID representing the product IDs of the sales.
7   */
8  public class Solution
9  {
10     public static int calculateDesktopProductIDs(char[] productID)
11     {
12         int answer = 0;
13
14         for (char c : productID)
15         {
16             if ("aeiouAEIOU".indexOf(c) == -1)
17             {
18                 answer++;
```

Code Analysis

Average-case Time Complexity

Candidate code: O(N)

Best case code: O(N)

*N represents number of products.

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

Readability & Language Best Practices

Line 14: Variables are given very short names.

© 2026 SHL and/or its affiliates. All rights reserved..

```
        }
    }

    return answer;
  }

  public static void main(String[] args)
  {
    Scanner in = new Scanner(System.in);

    int productID_size = in.nextInt();
    char productID[] = new char[productID_size];

    for (int idx = 0; idx < productID_size; idx++)
    {
      productID[idx] = in.next().charAt(0);
    }

    int result = calculateDesktopProductIDs(productID);
    System.out.print(result);
  }
}

```

Test Case Execution

Passed TC: **100%**

Total score

15/15

100%	100%	100%
Basic(**10**/10)	Advance(**4**/4)	Edge(**1**/1)

Compilation Statistics

3	3	0	0	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:18:23**
Average time taken between two compile attempts:	**00:06:08**
Average test case pass percentage per compile:	**100%**

Similarity Detected

Every response is checked against:

- The responses of peers who took this assessment in the same event
- All candidate responses submitted in the last week
- Content currently available on the web

This response has a high degree of similarity to one of these sources, which means it is possibly not the candidate's original work:

© 2026 SHL and/or its affiliates. All rights reserved..

Candidate Response	Peer's Response
7 */	7 */
8 public class Solution	8 public class Solution
9 {	9 {
10 public static int calculateDesktopProductIDs(char[] productID)	10 public static int calculateDesktopProductIDs(char[] productID)
11 {	11 {
12 int answer = 0;	12 int answer = 0;
13	13
14 for (char c : productID)	14 for (char ch : productID){
15 {	15 if("aeiouAEIOU".indexOf(ch) == -1){
16 if ("aeiouAEIOU".indexOf(c) == -1)	16 answer++ ;
17 {	
18 answer++;	
19 }	
20 }	17 }
21	18 }
	19
	20
22 return answer;	21 return answer;
23 }	22 }
24	23
25 public static void main(String[] args)	24 public static void main(String[] args)
26 {	25 {
27 Scanner in = new Scanner(System.in);	26 Scanner in = new Scanner(System.in);
28	28 //input for productID
29 int productID_size = in.nextInt();	28 int productID_size = in.nextInt();
30 char productID[] = new char[productID_size];	29 char productID[] = new char[productID_size];
31	31 for(int idx = 0; idx < productID_size; idx++)
32 for (int idx = 0; idx < productID_size; idx++)	
33 {	31 {

© 2026 SHL and/or its affiliates. All rights reserved..

```
        productID[idx] = in.next().charAt(0);
    }

    int result = calculateDesktopProductIDs(productID);
    System.out.print(result);

  }
}
```

```
        productID[idx] = in.next().charAt(0);
    }

    int result = calculateDesktopProductIDs(productID);
    System.out.print(result);

  }
}
```

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 2 (Language: Java)

A child's parent goes for a jog every morning. The child follows the parent several minutes later. The parent starts at a position that is X_1meters away from their home and runs in a straight line at a constant speed of V_1meters per step for N steps.

The child is standing X_2metres away from their home. They wonder how fast they must run at a constant speed of V_2metres per step to achieve a maximum F, where F equals the number of their parent's footsteps that the child will land on during their run. The first step that the child will land on from their starting position will have been landed on by their parent.

Note that, if more than one prospective speed results in the same number of maximum common steps, output the highest prospective speed as V_2.

Write an algorithm to calculate F and V_2.

© 2026 SHL and/or its affiliates. All rights reserved..

Scores

Programming Ability

20 / 100

Code seems to be unrelated to the given problem.

Programming Practices

75 / 100

Low readability, high on program structure. The source code does not follow best practices in its formatting and may contain a few redundant/improper coding constructs.

Functional Correctness

30 / 100

Partially correct basic functionality. The source code compiles and passes only some of the basic test cases. Some advanced or edge cases may randomly pass.

Final Code Submitted

Compilation Status: Pass

```
1  import java.util.*;
2
3  public class Solution
4  {
5     public static void main(String[] args)
6     {
7        Scanner in = new Scanner(System.in);
8
9        int parentPos = in.nextInt();
10       int childPos = in.nextInt();
11       int velParent = in.nextInt();
12       int steps = in.nextInt();
13
14       int F;
15       int V2;
16
17       if (velParent == 1)
18       {
19          F = steps + 1;
20          V2 = 1;
21       }
22       else
23       {
24          F = 1;
25          V2 = parentPos + velParent * steps - childPos;
26       }
27
28       System.out.print(F + " " + V2);
29    }
30 }
31
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code: O(N logN)

*N represents number of steps for which father runs.

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

Readability & Language Best Practices

Line 14,15: Variables are given very short names.
Line 14,15: variable starting with uppercase.

© 2026 SHL and/or its affiliates. All rights reserved..

Test Case Execution

Passed TC: **30%**

Total score 3/10

50%	0%	0%
Basic(**3**/6)	Advance(**0**/3)	Edge(**0**/1)

Compilation Statistics

15	13	2	0	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:26:36**
Average time taken between two compile attempts:	**00:01:46**
Average test case pass percentage per compile:	**4.67%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Automata Fix

100 / 100 Code Replay

Question 1 (Language: Java)

Lisa always forgets her birthday which is on the 5th of July. So, develop a function/method which will be helpful to remember her birthday.

The function/method ***checkBirthDay*** return '1' if it is her birthday else returns 0. The function/method ***checkBirthDay*** accepts two arguments - *month*, a string representing the month of her birthday and *day*, an integer representing the date of her birthday.

The function/method compiles successfully but fails to return the desired result for some test cases. Your task is to fix the code so that it passes all the test cases.

© 2026 SHL and/or its affiliates. All rights reserved..

Scores

Final Code Submitted

Compilation Status: Pass

```
class Solution
{
  int checkBirthDay(String month, int day)
  {
    if (month.equals("July") && day == 5)
      return 1;
    else
      return 0;
  }
}

```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Test Case Execution

Passed TC: **100%**

Total score

9/9

100%	100%	0%
Basic(**6**/6)	Advance(**3**/3)	Edge(**0**/0)

Compilation Statistics

1	1	0	0	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:01:57**
Average time taken between two compile attempts:	**00:01:57**
Average test case pass percentage per compile:	**100%**

© 2026 SHL and/or its affiliates. All rights reserved..

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 2 (Language: Java)

The function/method **selectionSortArray** accepts two arguments - *len* and *arr,* an integer representing the length of the list (0 < *len*) and a list of integers, respectively. It performs an in-place selection sort on the list and is supposed to return the list sorted in ascending order.

The function/method compiles successfully but fails to print the desired result for some test cases due to logical errors. Your task is to fix the code so that it passes all the test cases.

Assumption:
In this particular implementation of selection sort, the smallest element in the list is swapped with the element at first index, the next smallest element is swapped with the element at the next index and so on.

Scores

Final Code Submitted

Compilation Status: Pass

```
// You can print the values to stdout for debugging
class Solution
{
  public void selectionSortArray( int len, int[] arr )
  {
    int x = 0 , y = 0 ;
    for( x = 0 ; x < len ; x ++ )
    {
      int index_of_min = x;
      for( y = x ; y < len ; y ++ )
      {
        if (arr[index_of_min] > arr[y])

```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

© 2026 SHL and/or its affiliates. All rights reserved..

```
            {
                index_of_min = y;
            }
        }
        int temp = arr[x];
        arr[x] = arr[index_of_min];
        arr[index_of_min] = temp;
    }
  }
}
```

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Test Case Execution Passed TC: **100%**

Total score 9/9

100%	100%	0%
Basic(**4**/4)	Advance(**5**/5)	Edge(**0**/0)

Compilation Statistics

1	1	0	0	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:01:44**
Average time taken between two compile attempts:	**00:01:44**
Average test case pass percentage per compile:	**100%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 3 (Language: Java)

© 2026 SHL and/or its affiliates. All rights reserved..

The function/method ***allExponent*** returns the result of exponentiation of base raised to power exponent for all input values. The function/method ***allExponent*** accepts two arguments - *baseValue*, an integer representing the base and *exponentValue*, an integer representing the exponent.

The incomplete code in the function/method ***allExponent*** works only for positive values of the exponent. You must complete the code and make it work for negative values of exponent as well

The function/method ***allExponent*** uses another function/method - ***positiveExponent*** , return the exponentiation but accepts only positive exponent values. You are supposed to use this function to complete the code in ***allExponent*** function.
(Please refer to the Helper Code tab for details regarding these functions/methods).

Scores

Final Code Submitted **Compilation Status: Pass**

```
// You can print the values to stdout for debugging
class Solution
{
  float allExponent(int baseValue, int exponentValue)
  {
    float res = 1;
    if(exponentValue >=0)
    {
      Exponent exp = new Exponent(baseValue, exponentValue);
      res = (float)exp.positiveExponent();
    }
    else
    {
      res = 1 / (float)new Exponent(baseValue, -exponentValue).positiveExponent();

    }
    return res;
  }
}
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Test Case Execution Passed TC: **100%**

Total score 6/6

100%	100%	100%
Basic(**2**/2)	Advance(**3**/3)	Edge(**1**/1)

© 2026 SHL and/or its affiliates. All rights reserved..

Compilation Statistics

2	2	0	1	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:00:38**
Average time taken between two compile attempts:	**00:00:19**
Average test case pass percentage per compile:	**50%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 4 (Language: Java)

You are given a predefined structure Point and also a collection of related functions/methods that can be used to perform some basic operations on the structure.

You must implement the function/method ***isTriangle*** which accepts three points *P1, P2, P3* as inputs and checks whether the given three points form the vertices of a triangle.

If they form a triangle, the function/method returns 1. Otherwise, it returns 0.

(Please refer to the Helper Code tab for details regarding the structure Point and the predefined functions/methods around it).

Scores

Final Code Submitted **Compilation Status: Pass**

```
// You can print the values to stdout for debugging
```

Code Analysis

Average-case Time Complexity

© 2026 SHL and/or its affiliates. All rights reserved..

```
class Solution
{
   int isTriangle(Point P1, Point P2, Point P3)
   {
      if ((P1.x * (P2.y - P3.y) + P2.x * (P3.y - P1.y) + P3.x * (P1.y - P2.y)) != 0)
   return 1;
      else
   return 0;

   }

}
```

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Test Case Execution

Passed TC: **100%**

Total score

9/9

100%	100%	100%
Basic(**6**/6)	Advance(**2**/2)	Edge(**1**/1)

Compilation Statistics

1	1	0	0	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:01:02**
Average time taken between two compile attempts:	**00:01:02**
Average test case pass percentage per compile:	**100%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

© 2026 SHL and/or its affiliates. All rights reserved..

Question 5 (Language: Java)

The function/method **descendingSortArray** accepts two arguments - *len* and *arr,* an integer representing the length of the array and an array of integers, respectively. It performs an in-place selection sort on the list and is supposed to return the input list sorted in descending order, but instead, it returns the list sorted in ascending order due to logical errors in the code.

Your task is to debug the program to pass all the test cases.

The function/method compiles successfully, but instead, it returns the array sorted in ascending order due to logical errors in the code. Your task is to fix the code so that it passes all the test cases.

Scores

Final Code Submitted **Compilation Status: Pass**

```java
// You can print the values to stdout for debugging
class Solution
{
  public void descendingSortArray(int len, int[] arr)
  {
    int small, pos, i, j, temp;
    for( i = 0 ; i <= len - 1 ; i ++ )
    {
      for( j = i ; j < len ; j ++ )
      {
        temp = 0;
        if (arr[i] < arr[j])

        {
          temp = arr[i];
          arr[i] = arr[j];
          arr[j] = temp;
        }
      }
    }
  }
}
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Test Case Execution

Passed TC: **100%**

Total score 10/10

100%	100%	0%
Basic(**6**/6)	Advance(**4**/4)	Edge(**0**/0)

© 2026 SHL and/or its affiliates. All rights reserved..

Compilation Statistics

2	2	0	1	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:02:18**
Average time taken between two compile attempts:	**00:01:09**
Average test case pass percentage per compile:	**55%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 6 (Language: Java)

The function/method ***matrixSum*** returns the sum of elements of the input array matrix. The function/method ***matrixSum*** accepts three arguments - *rows*, an integer representing the number of rows of the input matrix, *columns*, an integer representing the number of columns of the input matrix and matrix, a two-dimensional array representing the input matrix.

The function/method ***matrixSum*** compiles unsuccessfully due to syntactical error. Your task is to debug the program so that it passes all test cases.

.

Scores

Final Code Submitted **Compilation Status: Pass**

```
// You can print the values to stdout for debugging
class Solution
{
```

Code Analysis

Average-case Time Complexity

© 2026 SHL and/or its affiliates. All rights reserved..

```
int matrixSum(int rows, int columns, int[][] matrix)
{
  int i, j, sum=0;
  for(i=0;i<rows;i++)
  {
    for(j=0;j<columns;j++)
      sum += matrix[i][j];

  }
  return sum;
}
}
```

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Test Case Execution

Passed TC: **100%**

Total score

10/10

100%	100%	0%
Basic(**6**/6)	Advance(**4**/4)	Edge(**0**/0)

Compilation Statistics

2	1	1	0	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:00:38**
Average time taken between two compile attempts:	**00:00:19**
Average test case pass percentage per compile:	**100%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

© 2026 SHL and/or its affiliates. All rights reserved..

Question 7 (Language: Java)

The function/method ***patternPrint*** accepts an argument *num*, an integer.
The function/method ***patternPrint*** prints *num* lines in the following pattern.

For example, *num* = 4, the pattern should be:
1
1 1
1 1 1
1 1 1 1

The function/method ***patternPrint*** compiles successfully but fails to print the desired result for some test cases due to incorrect implementation of the function/method. Your task is to fix the code so that it passes all the test cases.

Scores

Final Code Submitted **Compilation Status: Pass**

```
// You can print the values to stdout for debugging
class Solution
{
  void patternPrint(int num)
  {
    int print=1;
    for(int i=0;i<num;i++)
    {
      for(int j=0;j<=i;j++)
      {
        System.out.print(print+" ");
      }
      System.out.println();
    }
  }
}
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Test Case Execution

Passed TC: **100%**

Total score 8/8

100%	0%	100%
Basic(**7**/7)	Advance(**0**/0)	Edge(**1**/1)

© 2026 SHL and/or its affiliates. All rights reserved..

Compilation Statistics

2	1	1	0	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:11:43**
Average time taken between two compile attempts:	**00:05:52**
Average test case pass percentage per compile:	**100%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

© 2026 SHL and/or its affiliates. All rights reserved..

4 Learning Resources

The resources are not available for any of the tests taken by the candidate.

© 2026 SHL and/or its affiliates. All rights reserved..